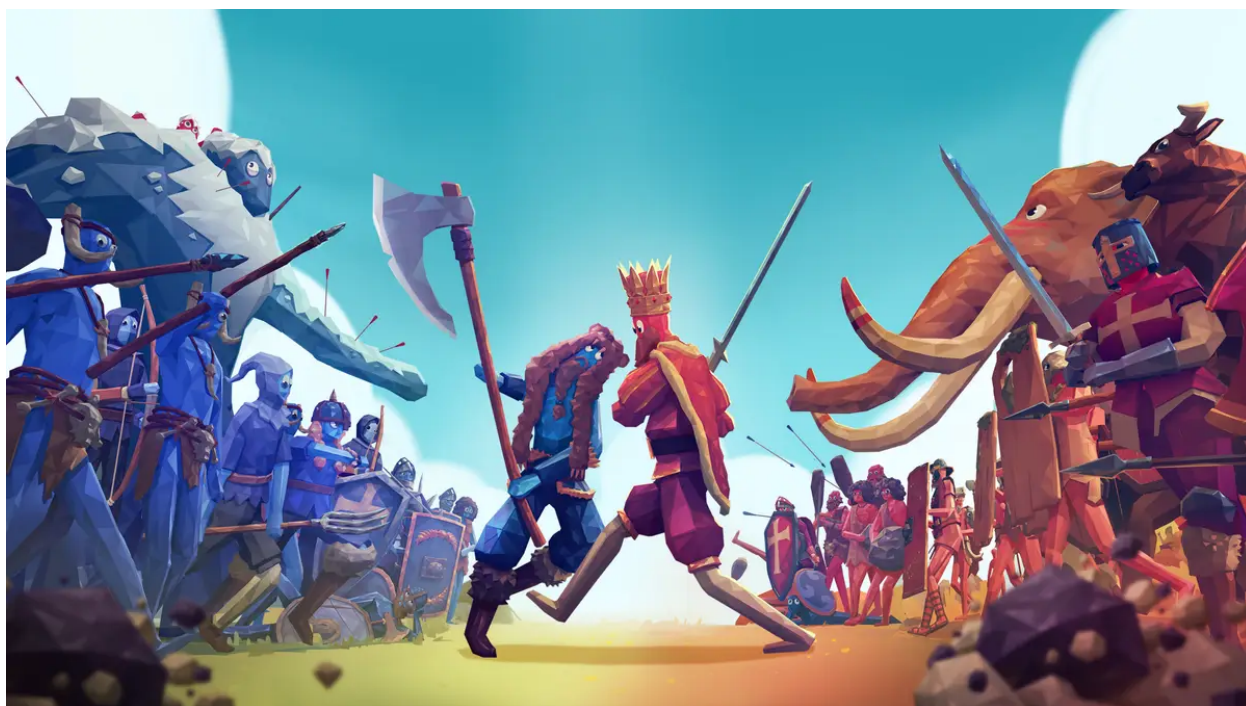


PHP OOP Mini-Project: RPG Battle Simulator



ზოგადი აღწერა:

ამ ამოცანაში თქვენ განავითარებთ მარტივ RPG Battle Simulator-ს PHP-ის გამოყენებით, ობიექტზე ორიენტირებული პროგრამირების (OOP) ძირითადი კონცეფციების გამოყენებით.

სიმულატორში მონაწილეობას მიიღებენ ისეთი პერსონაჟები, როგორიცაა **Warriors**, **Mages** და **Healers**, რომლებიც ებრძვიან ერთმანეთს, თითოეულს განსხვავებული შესაძლებლობები და თვისებები აქვს.

ეს არის პრაქტიკული პროექტი, რომელიც საშუალებას მოგცემთ ივარჯიშოთ სხვადასხვა OOP პრინციპებით, როგორიცაა მემკვიდრეობა, ინკაფსულაცია, თვისებები / პარამეტრები, ინტერფეისები, სტატიკური მეთოდები/თვისებები და პოლიმორფიზმი. სურვილისამებრ განავრცეთ თამაში.

ყურადღება უნდა გამახვილდეს სიმულატორისთვის PHP კლასების შემუშავებაზე და პროექტის გაშვებაზე პერსონალური Laravel Artisan ბრძანების მეშვეობით (`php artisan simulate:battle`). ეს ბრძანება მოახდენს თქვენს მიერ შექმნილ სხვადასხვა პერსონაჟებს შორის ბრძოლის სიმულაციას, ტერმინალში გამოაქვს ბრძოლის პროგრესი და შედეგი.

თქვენ უნდა შექმნათ

`Character` საბაზისო კლასი (ან აბსტრაქტული კლასი), რომელიც განსაზღვრავს საერთო თვისებებს, როგორიცაა `name` , `health` , და `attack power` ა.შ. ეს საბაზო კლასი უნდა შეიცავდეს მეთოდებს, რათა განისაზღვროს საერთო ქმედებები, როგორიცაა თავდასხმა ან ზიანის მიღება.

განახორციელეთ მინიმუმ სამი კონკრეტული პერსონაჟის ტიპის განმარტება (მაგ., `Warrior` , `Mage` , `Healer`), თითოეული უნიკალური ატრიბუტებითა და შესაძლებლობებით. გამოიყენეთ მემკვიდრეობა საბაზისო `Character` კლასის გასაგრძელებლად და დაამატეთ მეთოდები თითოეული პერსონაჟის განსხვავებული უნარების ასახვისთვის (მაგ., მეომრები იყენებენ ფიზიკურ შეტევებს, მოგვები აკეთებენ შელოცვებს).

გამოიყენეთ ინკაფსულაცია შესაბამისი თვისებების `private` ან `protected` მონიშვნით და ამ თვისებებზე წვდომის გასაკონტროლებლად `getters/setters` შექმნით.

გამოიყენეთ `traits` , რათა განისაზღვროს საერთო შესაძლებლობები ან ქცევები, რომლებიც შეიძლება გამოყენებულ იქნას პერსონაჟების სხვადასხვა ტიპებში (მაგალითად, მაგიური თვისება, რომელიც შეიძლება გამოიყენონ როგორც მოგვებმა, ასევე მკურნალებმა). თქვენ ასევე შეგიძლიათ გამოიყენოთ მრავალი `traits` ერთ კლასში და მოაგვაროთ მეთოდების კონფლიქტები საჭიროებისამებრ.

განსაზღვრეთ საბრძოლო სისტემა. ეს შეიძლება გაკეთდეს `Battle` კლასის შექმნით, რომელიც გაუმკლავდება მორიგეობით ბრძოლას პერსონაჟებს შორის. თქვენ ასევე უნდა შექმნათ ინტერფეისი (მაგ. `BattleInterface`), რომელიც განსაზღვრავს ბრძოლის სისტემის აუცილებელ მეთოდებს, როგორიცაა ბრძოლის დაწყება ან შეტევების განხორციელება.

აჩვენეთ პოლიმორფიზმი თქვენი პერსონაჟების კლასებში გაზიარებული მეთოდების დანერგვით, რაც საშუალებას გაძლევთ გამოიწვიოთ სხვადასხვა ქცევები გამოყენებული პერსონაჟების კლასის მიხედვით.

მაგალითად, `Attack()` მეთოდის გამოძახება `Warrior`-ზეც და `Mage`-ზეც განსხვავებული შედეგი უნდა იყოს, მიუხედავად იმისა, რომ მეთოდი ორივე კლასში არსებობს.

შექმენით სტატიკური თვისებები ან მეთოდები გლობალური ინფორმაციისათვის, როგორიცაა ბრძოლების მთლიანი რაოდენობა ან გარემო ფაქტორები, რომლებიც გავლენას ახდენენ თამაშზე (მაგ., რელიეფი ან ამინდის პირობები).

გამოიყენეთ კონსტრუქტორები პერსონაჟების ატრიბუტების ინიციალიზაციისთვის, როგორიცაა სახელი, ჯანმრთელობა და თავდასხმის ძალა ახალი ობიექტების შექმნისას.

დაამატეთ Laravel-ის მორგებული ბრძანება (`php artisan simulate:battle`), რომელიც მოახდენს თქვენს მიერ შექმნილ პერსონაჟებს შორის ბრძოლის სიმულაციას. ამ ბრძანებამ უნდა შექმნას მინიმუმ ორი პერსონაჟი, მოახდინოს მათი ურთიერთქმედების სიმულაცია და შედეგების გამოტანა ტერმინალში, მათ შორის ისეთი ქმედებები, როგორიცაა თავდასხმა, განკურნება და ბრძოლის საბოლოო შედეგი.

მინიმალური ძირითადი თამაშის მოთხოვნები:

- შექმენით `Character` საბაზისო კლასი და მინიმუმ სამი განსხვავებული პერსონაჟის ქვეკლასი უნიკალური ქცევით.
- განახორციელეთ ინკაფსულაცია, თვისებები და მემკვიდრეობა, რათა განისაზღვროს პერსონაჟის შესაძლებლობები და მოქმედებები.
- შექმენით ძირითადი `Battle` სისტემა, რომელიც სიმულაციას უკეთებს ორ ან მეტ პერსონაჟს შორის ბრძოლას.
- გამოიყენეთ მინიმუმ ერთი ინტერფეისი ბრძოლის მექანიკის დასადგენად
განახორციელეთ პოლიმორფიზმი სხვადასხვა პერსონაჟების ტიპების განსხვავებულად მოქმედებების შესრულებისას (მაგ., თავდასხმა).
- გამოიყენეთ სტატიკური თვისებები ან მეთოდები გლობალური მონაცემების თვალყურის დევნებისთვის (როგორიცაა ბრძოლების რაოდენობა...).

- შეიმუშავეთ პერსონალური Laravel Artisan ბრძანება (`php artisan simulate:battle`) თამაშის სიმულაციისთვის და შედეგების საჩვენებლად.

დემონსტრაცია (Console Output)



არ არის აუცილებელი იდენტური იყოს თქვენი პროექტი, ამას გიგზავნით მხოლოდ დემონსტრაციისათვის.

```
>> php artisan simulate:battle
```

```
===== RPG Battle Simulator =====
```

```
Starting battle between Aragorn (Warrior) and Gandalf (Mage)!
```

```
>> Round 1:
```

```
Aragorn (Warrior) attacks Gandalf (Mage) with a sword!
```

```
Gandalf receives 25 damage. Health remaining: 75
```

```
Gandalf (Mage) casts a fireball at Aragorn (Warrior)!
```

```
Aragorn receives 30 damage. Health remaining: 70
```

```
>> Round 2:
```

```
Aragorn (Warrior) strikes Gandalf with a powerful sword slash!
```

```
Gandalf receives 20 damage. Health remaining: 55
```

```
Gandalf (Mage) conjures a healing spell and restores 20 health.
```

```
Gandalf's health is now: 75
```

```
>> Round 3:
```

```
Aragorn (Warrior) lunges with a sword, dealing heavy damage!
```

```
Gandalf receives 35 damage. Health remaining: 40
```

Gandalf (Mage) retaliates with a lightning bolt!
Aragorn receives 40 damage. Health remaining: 30

>> Round 4:

Aragorn (Warrior) delivers a critical strike!
Gandalf receives 50 damage. Health remaining: -10

===== End of Battle =====

Gandalf (Mage) has been defeated!

Winner: Aragorn (Warrior)

Total turns: 4

Total damage dealt by Aragorn: 130

Total damage dealt by Gandalf: 70